

P13 Reports spec

Model: Opus, high effort.

Branch: none. Commit directly to `main`.

Depends on: P12.

1. What to do, in one sentence

Turn `docs/REPORT-SPEC.md` into an exact API contract for every report, R1 to R19, with one shared request and response shape, and update the report docs to match everything built since they were written, with **no code at all**.

2. Module

M19 Reports v2, the spec only. P14 builds the engine. P15 to P18 build the reports and their screens.

3. Why this is its own prompt

Reports are Caffeza's main requirement, and the rule is that they cannot be wrong.

`docs/REPORT-SPEC.md` was written before P02 to P11 existed. It says what each report shows. It does not yet say the exact field names, the exact response shape, or which stored field each column reads, because those fields did not exist yet. Now they do.

Writing the contract once, before P14 to P18, means five build sessions share one shape instead of inventing five.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P13-reports-spec.md`.
2. `git pull` . `git status` should show only that file.
3. `docs/prompts/README.md` must show P12 as Done. If not, stop.
4. No tests to run. If you find yourself changing a `.js` file, stop.

5. Files to read first

1. `docs/REPORT-SPEC.md` , `docs/GLOSSARY.md` , `docs/RECONCILIATION-RULES.md` and `docs/TEST-DATA.md` , all of them.
2. `docs/API-CONTRACT.md` : the "M6" section, for the principles that already apply, and the M8, M10, M16, M17 and M18 sections, for the stored fields.
3. `docs/DB-SCHEMA.md` sections 9, 12, 19 to 25.
4. Read, but do not change:
`server/services/reportRangeService.js` ,
`server/services/salesReportService.js` ,
`server/services/dayFiguresService.js` ,
`server/services/reconciliationService.js` ,

```
server/routes/reportRoutes.js ,  
client/src/features/reports/ReportShell.jsx .
```

6. What to write

A new section in `docs/API-CONTRACT.md` , "M19 Reports v2", with the parts below, in this order.

6a. Principles

Start by carrying over every M6 principle, by reference, so nobody has to choose between two rule lists: voided bills excluded everywhere, business date ranges only, `from` and `to` required and inclusive, every pipeline starting with the tenant `$match` , and the existing `MAX_RANGE_DAYS` of 366.

Then add the M19 rules from `docs/REPORT-SPEC.md` section 1: one source of frozen values, one engine, the filter sentence, open days, totals and averages, drill down, checks, export and formatting. Write each as a rule the code must follow, not as a description.

6b. The shared request

Every report endpoint takes the same query parameters where they apply:

Parameter	Meaning
<code>from</code> , <code>to</code>	Business dates, <code>YYYY-MM-DD</code> , inclusive. R1 and R2 take one <code>date</code> instead.

Parameter	Meaning
Filters	Only those listed for that report, for example orderType , platform , captainId , method , stationId
format	json , the default, or xlsx for the Excel download

Unknown parameters are refused with 400, the same as every validator in this repo.

6c. The shared response

Write one envelope that every report returns, and show it filled in for one real example, R3 for the golden day:

```
{
  "success": true,
  "data": {
    "report": "R3",
    "title": "Sales by Day",
    "filter": { "from": "2026-09-26", "to":
"2026-09-26" },
    "filterSentence": "26 Sep 2026. Business day
starts 5:00 AM. All order types. Voided bills
left out.",
    "openDays": [],
    "columns": [ { "key": "billTotalInPaise",
"label": "Bill total", "type": "money" } ],
    "rows": [ ],
    "totals": { },
    "checks": [ { "id": "C5.6", "severity":
"ERROR", "passed": true, "message": "..." } ],
    "generatedAt": "..."
  }
}
```

```
}  
}
```

Decide and write down:

1. The column `type` values and how each is shown:
`money` in paise, `count`, `percent` in basis points,
`text`, `date`, `time`, `minutes`.
2. Every `label` comes from `docs/GLOSSARY.md`. List, per report, the glossary term for every column.
3. How a cell carries its drill down. Suggested: each row may carry `drill: { [columnKey]: { report: "R19", query: { ... } } }`, and totals carry the same. Choose one shape and use it everywhere.
4. How sections work for reports with more than one table, like R2 and R15: a `sections` array, each with its own `columns`, `rows` and `totals`.
5. How `openDays` is worked out: business dates in the range with no `CLOSED` record in `dayclosures`.
6. How a check failure is shown: the result shape that `reconciliationService.js` already returns from P10.

6d. Each report

For each of R1 to R19, write:

1. The endpoint: `GET /api/v1/reports/{name}`.
Suggested names: `today`, `day-close`, `sales-by-day`, `hours`, `payments`, `platform-money`, `cash-till`, `gst`, `tally-export`, `invoice-register`, `menu`, `captains`, `tables`, `discounts`, `cancellations`, `no-charge`, `accounts`, `activity`,

`bills` . Where an M6 endpoint is being extended, as `docs/REPORT-SPEC.md` section 4 says for R3, R4 and R14, keep its path and add to it.

2. Its roles, from `docs/REPORT-SPEC.md` .
3. Its filters.
4. Its columns: key, glossary label, type, and **the stored field it reads**, naming the collection and field exactly, for example `bills.lines[].taxableInPaise` .
5. Its totals, and which are sums and which are averages.
6. Its checks, from the table in `docs/RECONCILIATION-RULES.md` section 3.
7. Its drill down target for each column.
8. One response example for R2, R5, R11 and R15 using the golden day's numbers from `docs/TEST-DATA.md` section 4. The other reports may describe their shape without a full example.

Specific points to settle while writing:

1. **R2** returns exactly `computeDayFigures` from P10, wrapped in the envelope. For a closed day it returns the stored snapshot, and C12 compares it with a fresh computation.
2. **R5** groups payments by their own `businessDate` , not the bill's.
3. **R6** is one row per payout batch, plus a section listing platform payments no live payout covers yet.
4. **R9** is a file only. Write the exact column layout of each sheet, matching the "Tally Data" blocks in `docs/REPORT-SPEC.md` R9, and the Tally codes from each payment's frozen `tallyLedgerCode` .

5. **R10** groups by `invoiceSeries` . A bill with a null series belongs to its `financialYear` series.
6. **R11** reads `bills.lines[]` , the frozen category and item names, and the line shares. Lines on bills from before P03 have no shares: count them in a separate "not recorded" row, never silently left out, never estimated.
7. **R12** groups by `captainId` , showing `captainName` from the most recent bill in the range.
8. **R15** reads cancelled lines from `orders` . A line cancelled as part of a whole-order cancel takes the order's `cancelReasonCode` , as P04 noted.
9. **R16** reads orders with status `NO_CHARGE` , by `noCharge.businessDate` .
10. **R18** is the M8 Audit Trail section as written, unchanged.
11. **R19** is the drill target. Its filters must be able to express every drill down any other report produces. Check this report by report, and list the result.

6e. Export

Write how `format=xlsx` works:

1. Built on the server from the same data the JSON response holds, never queried a second time.
2. Four sheets: the report, the filter sentence, the definitions of every column used, taken from the glossary, and the check results.
3. Money as numbers in rupees with two decimals and an Indian number format, so the file can be added up in Excel.

4. The file name: `{restaurant}-{report}-{from}-{to}.xlsx` , with spaces replaced.

5. The library: `exceljs` . It is added in P14.

PDF and print come from the browser's print function with an A4 print stylesheet, built in P18. No PDF library.

6f. Indexes

For each report, name the index its main query uses. Where none exists, write the index to add, starting with

`restaurantId` , and list them together. P14 adds them.

At least consider: payments by their own business date,

`orders` by `status` and `noCharge.businessDate` ,

cancelled lines by cancel time, `accountentries` by account and date, `platformpayouts` by method and period.

6g. Permissions

One table: every report endpoint against the six roles.

R5 and R7 stay OWNER only, as `docs/REPORT-SPEC.md` says, because they show expected cash.

7. Update the other report docs

1. `docs/REPORT-SPEC.md` : wherever a section describes a field that now has an exact name, use the name.

Replace section 6, "Fields the reports depend on", with a pointer to the new contract section, since every field now exists. Keep the reports, their questions and their rules exactly as they are, unless they contradict something built.

2. `docs/RECONCILIATION-RULES.md` : section 3, the report to check table, matches the contract.
3. `docs/GLOSSARY.md` : add any term the contract needs that is missing. Remove nothing.

Any contradiction you find between what was built and what these docs say: list it in your summary, and fix the doc to match what was built, unless what was built is wrong, in which case list it as a problem and do not fix it here.

8. Check your own work

Answer in your summary:

1. Can every number in `docs/TEST-DATA.md` section 4 be produced by one of the documented responses? Name the report and key for each.
 2. Does every column name a stored field, and does that field exist in `docs/DB-SCHEMA.md` ?
 3. Can R19 express every drill down from every other report?
 4. Does any report need a field that is not stored? If so, list it. Do not invent one.
-

9. Docs to update in PROJECT-STATE

1. The date line.
2. "Current stage": "Next: P14, the report engine."

3. Module status: M19 becomes IN PROGRESS, "Specified in P13."
 4. Decision log, dated today:

"Every M19 report returns one envelope: filter, filter sentence, open days, columns with glossary labels, rows, totals, checks and drill downs. Excel export is built on the server from the same data. | One shape for five build sessions, and the export can never disagree with the screen."

"R11 reports lines without stored shares as their own 'not recorded' row. | Old bills from before P03 must be visible, never silently dropped or estimated."
 5. "What changed recently": a P13 entry at the top, and the oldest moved to the archive.
 6. docs/prompts/README.md : mark P13 as Done.
-

10. Out of scope

Any code, any test, any model or index change. Specs only.
New reports beyond R1 to R19.
Charts. Their shape is decided in P18.

11. Done when

1. The M19 section is written, and the docs in section 7 and 9 are updated.
2. The four questions in section 8 are answered, and every gap is fixed or listed.

3. `npm run lint` still passes, as a check that no code was touched.
4. One commit on `main`: `add m19 reports spec`.
5. Push `main`.
6. Print a short summary: the endpoints and their roles as one table, the indexes to add, the answers to section 8, and every contradiction found.